Blatt 07: Mini-Projekt C++

Carsten Gips, BC George (HSBI)

1 Aufgabe

Entwickeln Sie in Ihrem 3er-Team gemeinsam einen Interpreter *oder* einen Compiler für C++.

Nutzen Sie als Implementierungssprache Java.[1] Sie sollen ANTLR zur Erstellung Ihres Lexers und Parsers einsetzen (Lernziel).

Entwickeln Sie verschiedene Eingabebeispiele in unterschiedlicher Komplexität, mit denen Sie Ihren Interpreter bzw. Compiler testen können.

1.1 Variante A: Interpreter

Der Interpreter muss den zu interpretierenden C++-Code aus einer Datei einlesen können.

Sie brauchen keine interaktive Interpretation implementieren, d.h. eine REPL ist nicht notwendig. Der Interpreter muss bei Bedarf an relevanten Stellen einen Log-Output erzeugen, um die Arbeitsweise des Programms nachvollziehen zu können. Das Arbeitsergebnis soll auf der Konsole ausgegeben werden.

1.2 Variante B: Compiler

Der Compiler muss den zu kompilierenden C++-Code aus einer Datei einlesen können.

Der Compiler soll aus dem eingegebenen C++-Code passenden gültigen Java-Code erzeugen und in eine Datei speichern. Definieren Sie ggf. nötige Hilfsbibliotheken, so dass man den generierten Code zusammen mit den Hilfsbibliotheken mit Java übersetzen und ausführen kann.

1.3 Sprachumfang

Sie sollen mindestens folgende C++-Konzepte unterstützen:

- Basisdatentypen: `bool`, **`int`**, **`char`**
- Variablen
- Arrays
- C++-Referenzen
- Zuweisungen und Expressions
- Kontrollfluss: **`if`**-`then`-**`else`**, **`while`**-Schleifen
- Funktionen (Definition, Deklaration, Aufrufe)
- Klassen (mit Attributen und Methoden)
- Einfach-Vererbung
- Polymorphie (dynamisch, statisch)
- Eingebaute Funktionen: `print_bool`, `print_int`, `print_char` (Ausgabe eines Werts des jeweiligen Typs auf der Konsole)

[1] Nach **Absprache** können Sie auch eine andere Implementierungssprache verwenden.

Beachten Sie bei der Umsetzung, dass Polymorphie in C++ etwas anders funktioniert als in Java.

Andere mit C++ verbundene Konzepte wie beispielsweise Präprozessor, Header-Files, Pointer, Templates, Sichtbarkeiten in Klassen, Trennung Deklaration/Implementierung bei Klassen (Trennung .h und .cpp) oder Initialisierungslisten[2] brauchen Sie nicht umsetzen.

Sie finden im CB-Repo einige Beispiele, die mindestens umgesetzt werden sollten und die Sie zum Testen Ihres Interpreters bzw. Compilers nutzen können. Beachten Sie, dass diese Sammlung nicht vollständig ist.

1.4 Projektvorstellung: Walk-Through statt Präsentation

Stellen Sie Ihr Projekt am Semesterende vor (Termine siehe Fahrplan und Ankündigung).

Jedes Team hat dafür 20 Minuten Zeit.

Gehen Sie dabei am Code durch Ihr Projekt und diskutieren Sie relevante Teile, mindestens aber:

- Grammatik
- AST
- Semantische Analyse
- Interpreter bzw. Compiler

Demonstrieren Sie die Funktionsfähigkeit mit Ihren C++-Codebeispielen.

Sie sollen keine Folien erstellen. Die Präsentation soll live in der IDE erfolgen.

2 Abgabeformat

Reichen Sie den als ZIP-Datei zusammengepackten Quellcode des Interpreters bzw. Compilers elektronisch über ILIAS ein.

3 Bewertungskriterien

1. **Inhalt (40 Punkte)**
   - **Aufgabenstellung (30 Punkte)**: Wurden alle Aspekte der Zielsprache sinnvoll umgesetzt?
   - **Argumentation und Nachvollziehbarkeit (10 Punkte)**: Sind die Konzepte logisch und schlüssig dargestellt? Werden die Aussagen durch relevante Code-Stellen gestützt?
2. **Verschiedenes (10 Punkte)**
   - **Roter Faden (5 Punkte)**: Wird der rote Faden während des gesamten Walk-Through beibehalten? Ist der Zusammenhang zwischen den einzelnen Punkten nachvollziehbar?
   - **Zeitmanagement (5 Punkte)**: Wurde der Zeitrahmen (20 Minuten pro Vortrag) eingehalten?

Gesamtbewertung: 50 Punkte

[2] Leider werden Initialisierungslisten für den Aufruf der Basisklassenkonstruktoren benötigt, sofern der Defaultkonstruktor der Basisklasse nicht ausreicht.

Unless otherwise noted, this work is licensed under CC BY-SA 4.0.

> **Last modified:** 9cb045f (readme: no more reference-links when formatting w/ pandoc, 2025-07-25)